

TECNOLÓGICO NACIONAL DE MÉXICO
INSTITUTO TECNOLÓGICO DE SALTILLO

Materia: Cómputo en la nube

Hora clase: 7:00am - 8:00am

Actividad: Github

Equipo 9

Cesar Gabriel Rodriguez Aguilera

No. Control: 21051501

Eliot Esau Renteria Rodriguez

No. Control: 21051498

Fecha: 23/02/2026

Actividad 2.4.1: Explorando GitHub

Conteste las siguientes preguntas y envíelas en formato PDF a Educatec. Para realizar esta

actividad se requiere una cuenta del GitHub, navegar por la plataforma, experimentar con

repositorios, relacionar lo aprendido con situaciones reales de trabajo colaborativo

1. ¿Qué diferencias encuentra entre un repositorio público y uno privado en GitHub, y en qué situaciones sería conveniente usar cada uno?

Tipo de Repo	Cuando usarlo	Ejemplo
Público	Proyectos de código abierto (<i>Open Source</i>), portafolios, aprendizaje público, herramientas colaborativas.	Desarrollar una librería pública, mostrar proyectos personales a reclutadores.
Privado	Código comercial, proyectos de empresa, información personal, prototipos iniciales.	Una aplicación SaaS, código con claves API, código fuente protegido.

2. Explique el proceso que siguió para crear su primer repositorio en GitHub y subir un archivo desde su computadora.

Para crear mi primer repositorio en GitHub seguí los siguientes pasos:

1. Ingresé a mi cuenta personal en la plataforma.
2. En la página principal, hice clic en el botón “New” o “New repository”.
3. Asigné un nombre al repositorio y agregué una breve descripción del proyecto.
4. Seleccioné si el repositorio sería público (visible para todos) o privado (solo accesible para colaboradores autorizados).
5. Opcionalmente, marqué la casilla para crear un archivo README inicial.
6. Finalmente, presioné el botón “Create repository” para generarlo.

Una vez creado el repositorio, para subir un archivo desde mi computadora realicé lo siguiente:

1. Dentro del repositorio, seleccioné la opción “Add file”.
2. Elegí “Upload files”.
3. Arrastré el archivo desde mi equipo o lo seleccioné manualmente.
4. Escribí un mensaje descriptivo del cambio en el apartado de *commit*.
5. Confirmé la acción haciendo clic en “Commit changes”.

Este proceso me permitió almacenar mi archivo en la nube, mantener un control de versiones y comenzar a organizar mi proyecto de forma profesional. Además, comprendí que cada archivo subido queda registrado mediante un *commit*, lo cual facilita el seguimiento de cambios y el trabajo colaborativo.

3. ¿Qué son los commits en GitHub y cómo describe la importancia de redactar mensajes claros al realizarlos?

Los commits en github son "puntos de guardado" o instantáneas inmutables que registran cambios específicos en el código de un repositorio, permitiendo rastrear la evolución, revertir errores y colaborar. Redactar mensajes claros es crucial para la documentación, la depuración ágil, la colaboración efectiva y la generación automática de registros de cambios (changelogs).

4. ¿Qué pasos siguió para clonar un repositorio a su equipo local y qué ventajas encuentra en este procedimiento?

Para clonar un repositorio a mi equipo local seguí los siguientes pasos:

1. Ingresé a la plataforma de GitHub e inicié sesión.
2. Entré al repositorio que desea clonar.
3. Hice clic en el botón verde “**Code**”.
4. Copié la URL del repositorio (HTTPS o SSH).
5. Abrí la terminal (o Git Bash) en mi computadora.
6. Ejecuté el comando:

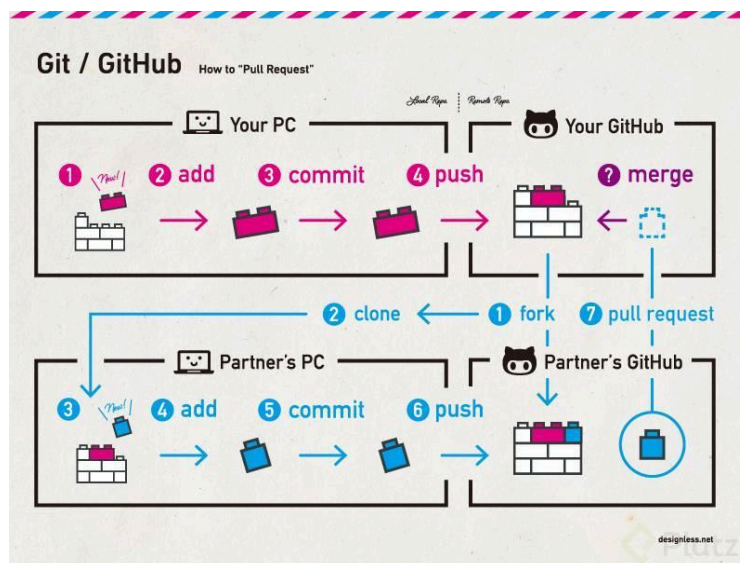
```
git clone <URL_DEL_REPOSITORIO>
```

7. Automáticamente se descargó una copia completa del repositorio en una carpeta local con todos sus archivos e historial de versiones.

5. Describa cómo funciona la opción de pull request y en qué tipo de proyectos colaborativos puede ser más útil.

El proceso de un pull request suele seguir estos pasos:

- Bifurcación o Ramificación (Branching): El desarrollador crea una rama separada (feature branch) para trabajar en una nueva funcionalidad o corrección de error sin alterar el código principal.
- Desarrollo y Commit: Se realizan los cambios en el código y se suben (hacen push) al repositorio compartido.
- Apertura del Pull Request: Se crea el PR en la plataforma, indicando qué cambios se quieren fusionar y desde dónde.
- Revisión y Discusión (Code Review): Otros miembros del equipo revisan el código, comentan líneas específicas, sugieren mejoras y discuten el enfoque.
- Ajustes: Si es necesario, el desarrollador hace cambios adicionales basados en la retroalimentación.
- Fusión (Merge): Una vez aprobado, el administrador del proyecto o un colaborador autorizado hace clic en "Merge pull request" para unir el código.



6. ¿Qué son las issues en GitHub y cómo pueden ayudarle a organizar y dar seguimiento a un proyecto?

Las issues (problemas) en GitHub son unidades de trabajo versátiles en un repositorio, diseñadas para reportar errores, solicitar nuevas funciones, gestionar tareas y fomentar debates. Funcionan como un sistema de seguimiento ligero que permite organizar, priorizar y asignar el desarrollo, facilitando la colaboración y la visibilidad del progreso del proyecto

¿Cómo ayudan a organizar y dar seguimiento a un proyecto?

- **Registro de trabajo centralizado:** Permiten documentar cada incidencia, tarea o idea con detalles, imágenes y comentarios, manteniendo todo el trabajo en un solo lugar.
- **Organización y Categorización:**
- **Asignación y Responsabilidad:** Se pueden asignar miembros específicos del equipo a cada *issue*, lo que aclara quién es responsable de cada tarea.
- **Seguimiento y Vinculación:**
- **Colaboración Activa:** Facilitan el debate mediante comentarios, menciones y reacciones para aclarar dudas y llegar a consensos sin salir de la plataforma.

7. Explique cómo se configura GitHub Pages para publicar un sitio web sencillo desde un repositorio.

1. Preparar el Repositorio

- **Crear un Repositorio:** Crea un nuevo repositorio en GitHub (público es recomendado para cuentas gratuitas).
- **Subir archivos:** Asegúrate de que tu sitio web contenga un archivo `index.html` en la raíz del repositorio. Este archivo será la página de inicio.
- **Subir el código:** Sube tu sitio mediante la terminal (`git push`) o directamente a través de la interfaz web de GitHub

2. Activar GitHub Pages en la Configuración

1. Navega a la página principal de tu repositorio en GitHub.
2. Haz clic en la pestaña Settings (Configuración) ubicada en la parte superior derecha.

3. En la barra lateral izquierda, busca la sección "Code and automation" (Código y automatización) y haz clic en Pages.
4. Debajo de "Build and deployment" (Construcción y despliegue), en "Source" (Fuente), selecciona Deploy from a branch (Desplegar desde una rama).
5. En la sección "Branch" (Rama):
 0. Selecciona la rama que quieres publicar (usualmente main o master).
 1. Selecciona la carpeta, usualmente /(root).
 2. Haz clic en Save (Guardar).

3. Visualizar el sitio

1. Tras guardar, aparecerá un mensaje en color amarillo indicando que el sitio se está procesando.
2. Espera unos minutos y refresca la página; GitHub mostrará un mensaje en color verde con la URL de tu sitio, que suele ser: <https://tuusuario.github.io>.

8. ¿Qué descubrió al explorar la sección de Insights de un repositorio y cómo podría

ayudar a un profesor o a un equipo de trabajo?

9. ¿Cómo se integran las acciones de GitHub (GitHub Actions) y qué aplicaciones prácticas tiene en proyectos en la nube?

Integración de GitHub Actions

- **Archivos YAML:** Se definen flujos de trabajo en el directorio `.github/workflows` del repositorio, lo que permite la configuración como código.
- **Eventos (Triggers):** Los flujos se activan automáticamente con eventos de GitHub como hacer un *push*, crear una solicitud de cambios (*pull request*) o etiquetar una versión (*release*).
- **Runners:** GitHub proporciona máquinas virtuales (Linux, Windows, macOS) para ejecutar las tareas (Jobs), o se pueden usar servidores propios.
- **Marketplace:** Se pueden integrar acciones preconstruidas por la comunidad para tareas comunes (ej. configurar Node.js, acceder a AWS).

Aplicaciones Prácticas en Proyectos en la Nube

1. **Despliegue Continuo (CD):** Despliegue automático de aplicaciones web o contenedores Docker a servicios en la nube (AWS S3/EC2, Azure App Service, Google Cloud Run) inmediatamente tras una actualización en la rama principal.
2. **Integración Continua (CI):** Ejecución automática de suites de pruebas unitarias, de integración y análisis de código estático (linters) en cada *pull request* para asegurar la calidad.
3. **Infraestructura como Código (IaC):** Automatizar el aprovisionamiento y actualización de recursos en la nube utilizando herramientas como Terraform o ARM templates.
4. **Gestión de Imágenes:** Compilación y subida (push) de imágenes Docker a registros (Docker Hub, Amazon ECR, Azure Container Registry).
5. **Automatización de Tareas de Repositorio:** Etiquetado automático de incidencias, comentarios en pull requests, o limpieza de ramas inactivas.

Componentes Principales

1. **Workflow:** El proceso automatizado completo.
2. **Job:** Conjunto de pasos ejecutados en el mismo runner.
3. **Action:** El comando individual que realiza la tarea

10. ¿Qué proyecto interesante encontró en la comunidad de GitHub y cómo cree que podría inspirar su propio aprendizaje o trabajo académico?

Encontré proyectos de código abierto relacionados con sistemas operativos y herramientas educativas. Estos proyectos inspiran a mejorar habilidades técnicas, colaborar con otros desarrolladores y aplicar conocimientos en proyectos académicos

Estos repositorios son referentes porque tienen miles o cientos de miles de estrellas y aportan valor a programadores de todos los niveles:

1. **codecrafters-io/build-your-own-x** – Guía para *reconstruir tecnologías desde cero* y aprender profundamente cómo funcionan. (∞ miles de estrellas)
2. **freeCodeCamp/freeCodeCamp** – Plataforma educativa con cursos gratuitos de programación, web, datos y más.

3. **sindresorhus/awesome** – Lista enorme de recursos *awesome* sobre muchos temas (aprender, frameworks, herramientas).
4. **public-apis/public-apis** – Lista de APIs públicas gratuitas que puedes usar en proyectos.
5. **huggingface/transformers** – Biblioteca líder para modelos de lenguaje e inteligencia artificial (*machine learning/AI*).
6. **Snailclimb/JavaGuide** – Guía de aprendizaje e entrevistas para desarrolladores Java.
7. **airbnb/javascript** – Guía de estilo para escribir JavaScript de forma profesional.